

# Giáo Án

Phiên bản web: <https://chief-heliotrope-d20.notion.site/Gi-o-n-1d1b9e967cbc80d68a77e486528a9fa5?pvs=4>

📌 Lưu ý cho giảng viên:

- **Tiếp nối bài học trước:** Hãy đảm bảo học viên đang sử dụng mã nguồn Todo App đã xây dựng ở bài học trước.
- **Backend Express.js bài học trước:** Mã nguồn Express.js backend từ bài trước vẫn được sử dụng.

## Authentication Trong React

### 1. Mục tiêu

- Học viên hình dung được quy trình **Authentication** trong React.
- Hiểu được khái niệm **Authorization**.
- Triển khai được **Authentication** vào dự án React.

### 2. Nội dung

#### 2.1 Giới thiệu Authentication và Authorization

- **Authentication (Xác thực):** Quá trình xác minh **đanh tính** của người dùng. Trả lời câu hỏi "**Bạn là ai?**". Ví dụ: Đăng nhập bằng username và password.
- **Authorization (Ủy quyền):** Quá trình xác định **quyền hạn** của người dùng sau khi đã xác thực. Trả lời câu hỏi "**Bạn được phép làm gì?**". Ví dụ: Chỉ admin mới được phép xóa todo.
- **Trong bài học này, chúng ta tập trung vào Authentication (xác thực đăng nhập/dăng xuất) với JWT.** Authorization (phân quyền) đa phần là do backend xử lý logic.

#### 2.2 Cơ chế Authentication với JWT (JSON Web Token)

- **JWT (JSON Web Token) là gì?**
  - Một chuẩn mở (open standard) để tạo ra **Access Tokens** cho việc Authentication và Authorization.
  - JWT là một chuỗi JSON được mã hóa (encoded), chứa thông tin (claims) về người dùng và được **ký số (digitally signed)** bởi server.
- **Quy trình Authentication với JWT (Đăng nhập & Đăng xuất):**
  - **Đăng nhập (Login):**
    - Người dùng nhập username và password vào form đăng nhập trên frontend.
    - Frontend gửi request đăng nhập (thường là POST) đến backend API ( `/api/login` ) kèm username và password.
    - Backend **xác thực thông tin đăng nhập** (ví dụ, kiểm tra username và password trong database).
    - Nếu xác thực thành công, backend **tạo ra một JWT Access Token** chứa thông tin về user (ví dụ, username).
    - Backend **trả về Access Token** cho frontend trong response.
    - Frontend **lưu trữ Access Token** (thường là trong Local Storage hoặc Cookies).
  - **Gọi API yêu cầu xác thực:**
    - Khi frontend cần gọi các API **yêu cầu xác thực** (ví dụ, lấy danh sách todos, thêm todo, ...), nó **gửi kèm Access Token** trong header `Authorization` của request (thường là `Bearer <accessToken>` ).
    - Backend **kiểm tra Access Token** trong header.
    - Nếu Access Token hợp lệ (chữ ký đúng, chưa hết hạn), backend **xác thực người dùng** và xử lý request.
    - Nếu Access Token không hợp lệ hoặc không có, backend **từ chối request** (trả về lỗi 401 Unauthorized hoặc 403 Forbidden).
  - **Đăng xuất (Logout):**

- Khi người dùng muốn đăng xuất, frontend **xóa Access Token** đã lưu trữ (ví dụ, xóa khỏi Local Storage hoặc Cookies).
- Các request API sau đó sẽ không còn gửi kèm Access Token nữa, và sẽ bị backend từ chối nếu yêu cầu xác thực.

## 2.3 Cập nhật code cho Backend

- **Cài đặt các dependencies mới:** Mở terminal trong thư mục backend và chạy `npm install jsonwebtoken`.
- **Cập nhật `server.js`:** Thay thế nội dung file `server.js` hiện tại bằng đoạn code backend đã cung cấp ở dưới.
- Giảng viên giới thiệu sơ qua các thay đổi backend cho học viên:

```
// server.js
const express = require('express');
const cors = require('cors');
const jwt = require('jsonwebtoken');

const app = express();
const port = 5000;

app.use(cors({
  origin: 'http://localhost:3000'
}));
app.use(express.json());

// Mã JWT Secret Key
const JWT_SECRET = 'LETDIV_BACKEND';

// Mô phỏng Database người dùng với dữ liệu mẫu
const users = [
  { username: 'admin', password: '123456' },
  { username: 'user1', password: 'password123' }
];

let todos = [
  { id: 1, text: 'Học React', completed: false },
  { id: 2, text: 'Làm bài tập Redux', completed: true },
  { id: 3, text: 'Thực hành gọi API', completed: false },
];

// Middleware xác thực JWT
const authenticateJWT = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (authHeader) {
    const token = authHeader.split(' ')[1];

    jwt.verify(token, JWT_SECRET, (err, user) => {
      if (err) {
        return res.sendStatus(403);
      }
      req.user = user;
      next();
    });
  } else {
    res.sendStatus(401);
  }
};

// API Đăng ký (Signup)
```

```

app.post('/api/signup', (req, res) => {
  const { username, password } = req.body;

  if (users.find(user => user.username === username)) {
    return res.status(409).json({ message: 'Username already exists' });
  }

  const newUser = { username, password };
  users.push(newUser);

  res.status(201).json({ message: 'User registered successfully' });
});

// API Đăng nhập (Login)
app.post('/api/login', (req, res) => {
  const { username, password } = req.body;

  const user = users.find(user => user.username === username);

  if (!user || user.password !== password) {
    return res.status(401).json({ message: 'Invalid username or password' });
  }

  const accessToken = jwt.sign({ username: user.username }, JWT_SECRET, { expiresIn: '1h' });

  res.json({ accessToken });
});

// API Lấy danh sách Todos
app.get('/api/todos', authenticateJWT, (req, res) => {
  res.json(todos);
});

// Các API POST, PATCH, DELETE Todos
app.post('/api/todos', authenticateJWT, (req, res) => {
  const newTodo = { ...req.body, id: Date.now(), completed: false };
  todos.push(newTodo);
  res.status(201).json(newTodo);
});

app.patch('/api/todos/:id', authenticateJWT, (req, res) => {
  const id = parseInt(req.params.id);
  const todoIndex = todos.findIndex(todo => todo.id === id);
  if (todoIndex > -1) {
    todos[todoIndex] = { ...todos[todoIndex], ...req.body };
    res.json(todos[todoIndex]);
  } else {
    res.status(404).json({ message: 'Todo not found' });
  }
});

app.delete('/api/todos/:id', authenticateJWT, (req, res) => {
  const id = parseInt(req.params.id);
  todos = todos.filter(todo => todo.id !== id);
  res.status(204).send();
});

app.listen(port, () => {

```

```
console.log(`Server listening at http://localhost:${port}`);
});
```

## 2.4 Triển khai đăng nhập/đăng ký

- Tạo `authSlice.js` :

```
// src/redux/slices/authSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import axiosClient from '../api/axiosClient';

export const register = createAsyncThunk(
  'auth/register',
  async ({ username, password }) => {
    const response = await axiosClient.post('/signup', { username, password });
    return response.data;
  }
);

export const login = createAsyncThunk(
  'auth/login',
  async ({ username, password }) => {
    const response = await axiosClient.post('/login', { username, password });
    const { accessToken } = response.data;
    localStorage.setItem('accessToken', accessToken); // Lưu access token vào localStorage
    return accessToken;
  }
);

export const logout = createAsyncThunk(
  'auth/logout',
  async () => {
    localStorage.removeItem('accessToken'); // Xóa access token khỏi localStorage
  }
);

const authSlice = createSlice({
  name: 'auth',
  initialState: {
    accessToken: localStorage.getItem('accessToken') || null, // Lấy token từ localStorage khi khởi tạo
    loading: false,
    error: null,
    user: null, // Optional: Có thể lưu thông tin user sau khi login thành công (nếu cần)
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(register.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(register.fulfilled, (state, action) => {
        state.loading = false;
        state.error = null;
      })
      .addCase(register.rejected, (state, action) => {
        state.loading = false;
        state.error = action.error.message || 'Registration failed';
      })
      .addCase(login.pending, (state) => {
```

```

        state.loading = true;
        state.error = null;
    })
    .addCase(login.fulfilled, (state, action) => {
        state.loading = false;
        state.accessToken = action.payload;
        state.error = null;
    })
    .addCase(login.rejected, (state, action) => {
        state.loading = false;
        state.error = action.error.message || 'Login failed';
        state.accessToken = null;
    })
    .addCase(logout.fulfilled, (state) => {
        state.accessToken = null;
        state.user = null;
    });
},
});

export default authSlice.reducer;

```

- Đăng ký authSlice vào `store.js` :

```

// src/redux/store.js
import { configureStore } from '@reduxjs/toolkit';
import authReducer from './slices/authSlice';
import todoReducer from './slices/todoSlice'; // Import todoReducer nếu có

const store = configureStore({
  reducer: {
    auth: authReducer,
    todos: todoReducer, // Thêm todoReducer vào reducer nếu có
  },
});

export default store;

```

- Tạo `Login.jsx` :

```

// src/screens/Login/Login.jsx
import React, { useState } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { Link, useNavigate } from 'react-router-dom';

import { login } from '../../redux/slices/authSlice';

function Login() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const dispatch = useDispatch();
  const loading = useSelector(state => state.auth.loading);
  const error = useSelector(state => state.auth.error);
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      await dispatch(login({ username, password })).unwrap();
    }
  }
}

```

```

        navigate('/');
    } catch (err) {
        console.log(error);
    }
};

return (
    <form onSubmit={handleSubmit}>
        <h2>Login</h2>
        {error && <p style={{ color: 'red' }}>{error}</p>}
        <div>
            <label htmlFor="username">Username:</label>
            <input
                type="text"
                id="username"
                value={username}
                onChange={(e) => setUsername(e.target.value)}
                disabled={loading}
            />
        </div>
        <div>
            <label htmlFor="password">Password:</label>
            <input
                type="password"
                id="password"
                value={password}
                onChange={(e) => setPassword(e.target.value)}
                disabled={loading}
            />
        </div>
        <button type="submit" disabled={loading}>
            {loading ? 'Logging in...' : 'Login'}
        </button>
        <p>
            Don't have an account? <Link to="/register">Register here</Link>
        </p>
    </form>
);
}

export default Login;

```

- Tạo `Register.jsx` :

```

// src/screens/Register/Register.jsx
import React, { useState } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { useNavigate, Link } from 'react-router-dom';

import { register } from '../../redux/slices/authSlice';

function Register() {
    const [username, setUsername] = useState('');
    const [password, setPassword] = useState('');
    const dispatch = useDispatch();
    const loading = useSelector(state => state.auth.loading);
    const error = useSelector(state => state.auth.error);
    const navigate = useNavigate();

```



```

const handleSubmit = async (e) => {
  e.preventDefault();
  try {
    await dispatch(register({ username, password })).unwrap();
    alert('Registration successful! Please log in.');
```

[navigate\('/login'\)](/login);

```

  } catch (err) {
    console.error("Login failed:", err);
  }
};

return (
  <form onSubmit={handleSubmit}>
    <h2>Register</h2>
    {error && <p style={{ color: 'red' }}>{error}</p>}
    <div>
      <label htmlFor="reg-username">Username:</label>
      <input
        type="text"
        id="reg-username"
        type="text"
        value={username}
        onChange={(e) => setUsername(e.target.value)}
        disabled={loading}
      />
    </div>
    <div>
      <label htmlFor="reg-password">Password:</label>
      <input
        type="password"
        id="reg-password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        disabled={loading}
      />
    </div>
    <button type="submit" disabled={loading}>
      {loading ? 'Registering...' : 'Register'}
    </button>
    <p>
      Already have an account? <Link to="/login">Login here</Link>
    </p>
  </form>
);
}

export default Register;

```

- Cập nhật `App.jsx`:

```

// src/App.jsx
import React from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';

// Import screen components
import Home from './screens/Home/Home';
import Login from './screens/Login/Login'; // Import Login screen
import Register from './screens/Register/Register'; // Import Register screen

```



```
function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/login" element={<Login />} /> {/* Thêm mới */}
        <Route path="/register" element={<Register />} /> {/* Thêm mới */}
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

- Tạo `LogoutButton.jsx` :

```
// src/components/LogoutButton/LogoutButton.jsx
import React from 'react';
import { useDispatch } from 'react-redux';
import { logout } from '../redux/slices/authSlice';

function LogoutButton() {
  const dispatch = useDispatch();
  const handleLogout = () => {
    dispatch(logout());
  };

  return (
    <button onClick={handleLogout}>Logout</button>
  );
}

export default LogoutButton;
```

- Cập nhật `Home.jsx` :

```
// src/screens/Home/Home.jsx
import React from 'react';

import TodoInput from '../components/TodoInput/TodoInput';
import TodoList from '../components/TodoList/TodoList';
import LogoutButton from '../components/LogoutButton/LogoutButton';

function Home() {
  return (
    <div>
      <div>
        <h1>Ứng Dụng Todo với Redux Toolkit</h1>
        <LogoutButton />
      </div>
      <TodoInput />
      <TodoList />
    </div>
  );
}

export default Home;
```

## 2.5 Đính kèm Access Token



- **Gửi Access Token trong Request Headers:**

- Khi frontend cần gọi API endpoint yêu cầu xác thực, cần đính kèm Access Token (JWT) vào header của request.
- **Header `Authorization`** : Sử dụng header `Authorization` với schema `Bearer` .
- Cập nhật `axiosClient.js` :

```
// src/api/axiosClient.js
import axios from 'axios';
import store from '../redux/store'; // **Nhập store Redux của bạn**

const axiosClient = axios.create({
  baseURL: process.env.VITE_API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Interceptor request - Thêm JWT vào header cho mỗi request (nếu có sẵn trong Redux store)
axiosClient.interceptors.request.use(
  (config) => {
    // **Lấy accessToken từ Redux store**
    const accessToken = store.getState().auth.accessToken;

    if (accessToken) {
      config.headers.Authorization = `Bearer ${accessToken}`;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);

export default axiosClient;
```

- **Backend xác thực JWT:**

- Như đã trình bày ở phần Backend ví dụ, backend cần middleware ( `authenticateJWT` trong ví dụ) để kiểm tra JWT trong header `Authorization` của mỗi request đến các API endpoint được bảo vệ.
- Middleware này sẽ:
  - Kiểm tra sự tồn tại của header `Authorization` và định dạng "Bearer <token>".
  - Trích xuất token.
  - Xác minh token (chữ ký, thời gian hết hạn) bằng secret key.
  - Nếu token hợp lệ, cho phép request đi tiếp và có thể lưu thông tin user từ payload vào `req.user` .
  - Nếu token không hợp lệ, trả về lỗi 401 hoặc 403.

## 2.6 Protected Routes

- **Tại sao cần Protected Routes?**

- Để hạn chế truy cập vào một số trang hoặc chức năng của ứng dụng chỉ dành cho người dùng đã đăng nhập.
- Ví dụ: Trong ứng dụng Todo này, chúng ta muốn **bảo vệ trang chính hiển thị danh sách Todo (trang Home)**, chỉ cho phép người dùng đã đăng nhập xem và thao tác với danh sách Todo của họ.

- **Cách triển khai Protected Route:**

- Tạo `ProtectedRoute.jsx` :

```
// src/components/ProtectedRoute/ProtectedRoute.jsx
import React from 'react';
import { useSelector } from 'react-redux';
import { Navigate } from 'react-router-dom'; // React Router v6

function ProtectedRoute({ children }) {
  // Lấy accessToken trực tiếp từ Redux state
  const accessToken = useSelector(state => state.auth.accessToken);

  if (!accessToken) {
    // Nếu không có accessToken (chưa đăng nhập), chuyển hướng đến trang đăng nhập
    return <Navigate to="/login" replace />;
  }

  // Nếu có accessToken (đã đăng nhập), render các component con (children) - trang được bảo vệ
  return children;
}

export default ProtectedRoute;
```

- Sử dụng `ProtectedRoute` trong `App.jsx`:

```
// src/App.jsx
import React from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Home from './screens/Home/Home'; // Import Home screen
import Login from './screens/Login/Login'; // Import Login screen
import Register from './screens/Register/Register'; // Import Register screen
import ProtectedRoute from './components/ProtectedRoute/ProtectedRoute'; // Import ProtectedRoute

function App() {
  return (
    <BrowserRouter>
      <Routes>
        {/* Các route công khai (không yêu cầu đăng nhập) */}
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />

        {/* Route được bảo vệ - Trang Home */}
        <Route
          path="/"
          element={
            // Bọc component Home bằng ProtectedRoute
            <ProtectedRoute>
              <Home />
            </ProtectedRoute>
          }
        />

        {/* Bạn có thể thêm các route được bảo vệ khác nếu cần */}

      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

## 2.7 Xử lý trường hợp có Access Token nhưng hết hạn

- Nếu đang có access token nhưng đã hết hạn thì React sẽ tự động điều hướng đến trang Login:
- Cập nhật `axiosClient.js` :

```
// src/api/axiosClient.js (Ví dụ - Interceptor xử lý lỗi 401 đơn giản)
import axios from 'axios';
import store from '../redux/store';
import { logout } from '../redux/slices/authSlice'; // Import logout action

const axiosClient = axios.create({
  baseURL: process.env.VITE_API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Request interceptor (giữ nguyên)
axiosClient.interceptors.request.use(config => {
  const accessToken = store.getState().auth.accessToken;
  if (accessToken) {
    config.headers.Authorization = `Bearer ${accessToken}`;
  }
  return config;
}, error => {
  return Promise.reject(error);
});

// Response interceptor (Cập nhật để phân biệt lỗi 401)
axiosClient.interceptors.response.use(
  response => response,
  error => {
    const { response, config: originalRequest } = error; // Lấy response và config của request gốc

    // Kiểm tra nếu lỗi là 401 VÀ request gốc KHÔNG PHẢI là request đăng nhập
    if (response && response.status === 401 && originalRequest.url !== '/login') {
      // Chỉ dispatch logout nếu lỗi 401 KHÔNG đến từ endpoint /login
      console.warn("Access Token expired or invalid during authenticated request. Logging out.");
      store.dispatch(logout());
      // ProtectedRoute sẽ xử lý việc chuyển hướng dựa vào state thay đổi
    }

    return Promise.reject(error);
  }
);

export default axiosClient;
```